Loops, Logic and Regular Expressions (RegEx)

Module VII

Logic and Loops

That’s loopy!

- Looping through a list of objects is a programming fundamental.
- Looping is used to ‘iterate’ a single (or multiple task) many times.
- In PWSH looping is:
  - used when we want to iterate a command against multiple objects
  - used to iterate a set of tasks until a condition is met to exit the loop
  - used when we want count or ‘increment’ a value
  - used to create menus for interactive scripts

(this list is not exhaustive)

Types of loops in PWSH

- `For` loop
- `ForEach` loop
- `while` loop
- `Do while` loop
- `Do until` loop
- Bonus! `(dot).notation` ← This *technically* loops too

For loop

For loop

- A For loop is a simple loop. It loops as long as a condition is $true.
- If the condition is $false, the loop ends.
- Syntax:

```
for (<Init>; <Condition>; <Repeat>) {
    <Statement list>
    }
```

- <Init> → one or more commands that runs before the loop begins
- <condition> → the expression that resolves to $true or $false
- <repeat> → one or more commands that executes each time the loop repeats (usually an incrementor)
- <statement list> → commands that repeat as the loop iterates

For loop example: counting!

```
for ($i = 1;  $i -lt 10; $i++){
    Write-Host $i
}
```

1. `$i = 1` → initialize `$i` as '`1`'
2. `$i -lt 10` → if `$i` is greater than or equal to '`10`', stop the loop
3. `$i++` → a common expression that increments an integer by '`1`'. Every time the loops runs, `$i` is incremented by '`1`'.
4. `Write-Host $i` → each time the loops is run (if `$i < 10 = $true`) write the current value of `$i` to the console

For loop (first iteration)

```
1 for ($i = 0; $i -lt 10; $i++){
2
3
4     Write-Host $i
5
6
7
8
9 }
```

During the first iteration, $I = 0

1

2 Is $i < 10? $true!

3 Increment $i by 1. $i = 1

4 Write “0” to the screen.

Start the loop over.

For loop (second iteration)

```
1  for ($i = 0; $i -lt 10; $i++){
2
3
4      Write-Host $i
5
6
7
8
9  }
```

During the 2nd iteration, $I = 1

1

2 Is $i < 10? $true!

3 Increment $i by 1. $i = 2

4 Write “1” to the screen.

Start the loop over.

For loop (11th iteration)

```
1  for ($i = 0; $i -lt 10; $i++){
2
3
4      Write-Host $i
5
6
7
8
9  }
```

During the 11th iteration, $I = 10

1

2 Is $i < 10? $false!

3

Exit the loop

ForEach, ForEach-Object

`ForEach` vs `ForEach-Object`

- These are *two separate commands*.
- `ForEach`
  - Iterate through the values in a collection or and array
  - Used to iterate through multiple objects
- `ForEach-Object`
  - Perform an operation defined in a script block on each item in the input collections objects
  - Used to when passing data through the pipeline.

ForEach

```
foreach ($item in $collection){
    <statement list>
}
```

- `$item` → a temporary variable
- `$collection` → the object we a referencing in the loop
- `<statement list>` → Do something here, every time we loop

ForEach

```
1   $dwarves = 'Doc','Grumpy','Happy','Sleepy','Bashful','Sneezy','Dopey'
2   foreach ($dwarf in $dwarves){
3       Write-host "$dwarf Dwarf"
4   }
```

- `$dwarves` → an array of items.
- `$dwarf` → a temporary variable that will 'capture' and store each value from the input array, one at a time, as the loop continues.
  - During the first iteration of the loop `$dwarf = 'Doc'`
  - During the second iteration of the loop `$dwarf = 'Grumpy'`
- `Write-host "$dwarf Dwarf"` → Write the name of each value to the screen each time the loop loops; append the string 'Dwarf' to the resulting output.

ForEach (first iteration)

```
7   $dwarves = 'Doc','Grumpy','Happy','Sleepy','Bashful','Sneezy','Dopey'
8
9
10  foreach ($dwarf in $dwarves){
11
12
13
14      Write-host "$dwarf Dwarf"
15
16
17
18  }
```

1 During the first iteration, $Dwarves = 'Doc'

2 During the first iteration $dwarf = $dwarves = 'Doc'

3 String = "$dwarf Dwarf" = "Doc Dwarf"

Start the loop over.

Write "Doc Dwarf" to the screen

ForEach (second iteration)

```
7   $dwarves = 'Doc','Grumpy','Happy','Sleepy','Bashful','Sneezy','Dopey'
8
9
10  foreach ($dwarf in $dwarves){
11
12
13
14      Write-host "$dwarf Dwarf"
15
16
17
18  }
```

1 During the second iteration, $Dwarves = ‘Grumpy’

2 During the second iteration $dwarf = $dwarves = ‘Grumpy’

3 String = “$dwarf Dwarf” = “Grumpy Dwarf”

Write “Grumpy Dwarf” to the screen

(Once all objects in the collection are processed, the loop exits)

ForEach-Object

```
1   $inputCollection | ForEach-Object { <statment list> }
```

- `$inputCollection` → What is being passed from the pipeline. This can be *a lot* of different things, as long as what is being piped is a collection of objects.
- `<statement list>` → Do something with `$_` (`$PSITEM`)

ForEach-Object

```
5   Import-Csv .\dwarves.csv | ForEach-Object { Enable-ADAccount -Identity $_.Dwarf}
```

- `Import-Csv .\dwarves.csv |` → Import the CSV file containing a collection of dwarf names. Put it in the pipeline.
- `ForEach-Object {Enable-ADAccount -Identity $_.Dwarf}` → Enable each active directory account that matches a value from the 'Dwarf' (`$_.Dwarf`) column in our collection.

ForEach-Object (first iteration)

.\dwarves.csv is processed by Import-CSV and a collection is produced. That collection is put in the pipe.

```
Import-Csv .\dwarves.csv |
    ForEach-Object { Enable-ADAccount -Identity $_.Dwarf}
```

The ForEach-object command processes each object in the collection one-by-one. The first $PSITEM is “Doc”.

1

2

“Doc” is passed to the –Identity parameter Enable-ADAccount

3

User is enabled in AD.

4

Start the loop over.

ForEach-Object (second iteration)

```
7   .\dwarves.csv is processed by Import-CSV and a collection is
8   produced. That collection is put in the pipe.
9   Import-Csv .\dwarves.csv |
10
11
12  ForEach-Object { Enable-ADAccount -Identity $_.Dwarf}
13
14
15
16
17
```

The ForEach-object command processes each object in the collection one-by-one. The first $PSITEM is “Grumpy”.

1

2

3 User is enabled in AD.

“Grumpy” is passed to the –Identity parameter Enable-ADAccount

4 Start the loop over.

(Once all objects in the collection are processed, the loop exits)

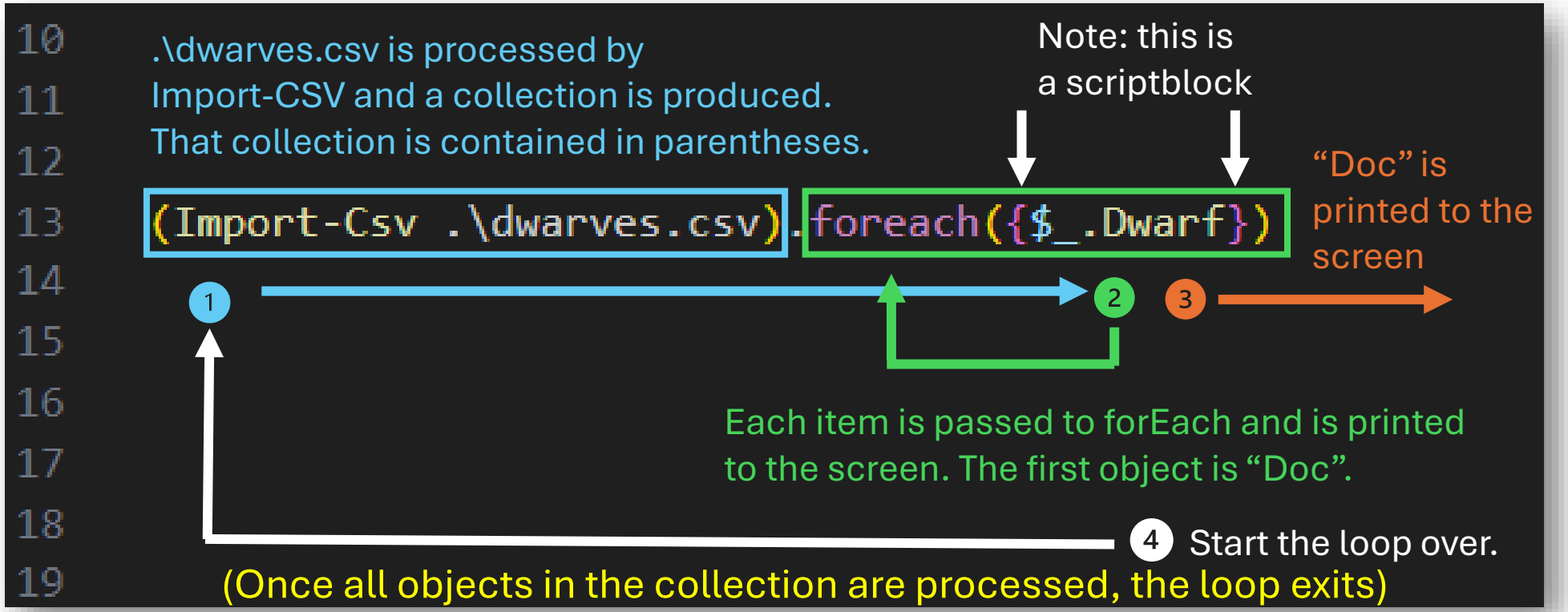
`ForEach` method

- `ForEach` can also be used as a method.

```
10    (Import-Csv .\dwarves.csv).foreach({$_.Dwarf})
```

```
10    (Import-Csv .\dwarves.csv).foreach({$_.Dwarf}).toupper()
```

ForEach method



While, Do While

`while` Loop

- An iterative loop that will run until a terminating condition is met
- Has a script block, like a `For` loop.

```
while (condition) {
    <statement list>
}
```

- `(condition)` → a Boolean statement that executes while `$true` and terminates when `$false`.
- `<statement list>` → Do something while condition = `$true`

while Loop

```
1   $n=1
2   While ($n -le 10){Write-Output $n; $n++}
```

- $n=1 → Initialize $n as '1'
- $n -le 10 → the condition to terminate the loop. If $n is less than or equal to 10, continue the loop
- Write-Output $n→ while the above condition is true, write $n to the screen
- $n++ → increment $n every iteration of the loop

while Loop (first iteration)

```
1   $n=1
2
3
4   While ($n -le 10){
5
6
7
8       Write-Output $n; $n++
9
10
11
12  }
```

1 $n is initialized as 1

2 While is $n <= 10, execute the statements in the scriptblock

3 Write “1” to the screen.

4 Start the loop over. Increment $n by 1

`while` Loop (final iteration)

```
1   $n=1
2
3
4   While ($n -le 10){
5
6
7
8       Write-Output $n; $n++
9
10
11
12  }
```

1 $n is initialized as 11 from the previous iteration

2 While is $n <= 10, execute the statements in the scriptblock. However, $n = 11, exit the loop.

Do while/until

```
Do {commands} While (condition)

Do {commands} Until (condition)
```

- `Do {commands}` → Execute the following statements
- `While (condition)` → Stop executing the above statements while a condition is evaluated.
- `Until (condition)` → Stop executing the above statements until a condition is evaluated.
- While/Until changes the logic in our loop

Do while/until

```
$number = 1

do {
    Write-Host "The variable is $number"
    $number++
} while ($number -le 10)
```

- `$number = 1` → initialize the variable as '`1`'
- `Write-Host "The variable is $number "` → Write the value of `$number` each time the loop is iterated.
- `$number++` → Increment the `$number` variable each time the loop is iterated.
- `while ($number -le 10)` → while the value of `$number` is less than or equal to 10 continue the loop. If `$number` is greater than `10`, stop the loop.

Do while/until

```
1  $number = 1
2
3  do {
4
5
6
7      Write-Host "The variable is $number"
8
9      $number++
10
11
12
13 } while ($number -le 10)
```

1 $number is initialized as 1

2 Execute the statements in the scriptblock

2 "The variable is 1 is printed to the screen

Increment $number by 1. $number now = 2

3

If $number is <= 10 start the loop over. If number is >10, Exit the loop

4

Do while/until

```
9   $number = 1
10
11  do {
12      Write-Host "The variable is $number"
13      $number++
14  } until ($number -ge 11)
```

- `$number = 1` → initialize the variable as '`1`'
- `Write-Host "The variable is $number"` → Write the value of `$number` each time the loop is iterated.
- `$number++` → Increment the `$number` variable each time the loop is iterated.
- `until ($number -ge 10)` → while the value of `$number` is greater than or equal to 10 continue the loop. If `$number` is greater than `10`, stop the loop.

`while` VS `Do while/Until`

- Use `while` when you do not want your sriptblock to execute if the condition is false.
- Use `do while` when you want your scriptblock to always execute at least once, even if the condition is false.

If $n is > 10, the script block will never execute.
The Value of $n is checked before the scriptblock executes

```
While ($n -le 10){
    Write-Output $n
    $n++
}
```

The script block execute at least once.
$n is check following the first iteration.

```
do {
    Write-Host "The variable is $number"
    $number++
} while ($number -le 10)
```

If $n is > 10, the scriptblock will not execute a second time.

Conditional Logic

If/Else

- `If` executes a statement based on the state of a condition.

```
if (condition) {
    <statment list>
}
```

- `(condition)` → check if this condition is true to determine if I should run the following statements
- `<statement list>` → commands to run if the condition is true

If

```
[int32]$dogs = Read-Host "How many dogs do you have?"

if ($dogs -ge 1) {
    Write-Output "You have at least one dog.  Infact you have $dogs dogs!"
}
```

- `[int32]$dogs = Read-Host "How many dogs do you have?"`→ Capture the value of `$dogs` using user input.
- `$dogs -ge 1` → if `$dogs` is greater than `0`, execute the following statements
- `Write-Output "You have at least one dog. In fact, you have $dogs dogs!"`
- → Write this output message to the screen if the condition was true

If/Else

- `If` statements are often accompanied by an `Else` statement.
- `Else` statements allows you to execute other statements if the condition that is evaluated is `$false`.

If/Else

```
if (condition) {
    <statment list if true>
} else {
    <statment list if false>
}
```

- `(condition)` → check if this condition is true to determine if I should run the following statements
- `<statement list if true>` → commands to run if the condition is true
- `<statement list if false>` → commands to run if the condition is false

If/Else

```
[int32]$dogs = Read-Host "How many dogs do you have?"

if ($dogs -ge 1) {
    Write-Output "You have at least one dog.  Infact you have $dogs dogs!"
} else {
    Write-Output "You have have no dogs :("
}
```

- `$dogs -ge 1` → if `$dogs` is greater than `0`, execute the following statements
- `Write-Output "You have at least one dog.  In fact, you have $dogs dogs!"`
  → Write this output message to the screen if the condition was true
- `Write-Output "You have no dogs :("`
  → Write this output message to the screen if the condition was flase

Regular Expressions (RegEx)

What is RegEx and do I need it?

- Regular Expressions or RegEx is a method of parsing text and text files.
- While it is completely necessary to know RegEx for Unix Admins, Powershell admins rely less on RegEx because most of our data is *not typically text*.
- RegEx is still very helpful for administrators. The best practical application of this would be parsing log files.

The purpose of a Regular Expression

- RegEx is written for a specific language. So, while Unix also uses RegEx, the expressions *use a different syntax*.
- RegEx can be used to Validate data.
  - How would we indicate that the IP address 500.234.543.544 is an illegal value even though its in the correct format?
  - RegEx can define **patterns of numbers**.
  - RegEx can define **patterns of text**.
- RegEx could be used to find a specific error code in a log file, for example.

As easy as I can make it…

- Let's look at a *simple* regular expression: “`tough`”
- By default, which of the following expressions would match?
  - `TOUGH`
  - `tough`
  - `tOugh`
  - `TougH`
  - `TOuGh`
- **They all would**. Be default PowerShell's expression matching is case-*insensitive*.

Meet your match…

- We can instruct Powershell to look for different patterns to match.
- Certain character sets have special meaning in PWSH.
- Where are we going to use these RegEx's (once we get good at them)?
  - `Select-String`
  - `-match` and `-replace` operators
  - `-split`
- PowerShell regular expressions are case-insensitive by default. Each method shown above has a different way to force case sensitivity.
  - `Select-String` → use `-CaseSensitive` switch
  - Operators → prefix with 'C' (`-cmatch`, `-csplit`, or `-creplace`)

Types of RegEx

Character literals

- A regular expression can be a literal character or a string. The expression causes the engine to match the text specified **exactly**.
- This statement returns true because book contains the string "oo"
  - `'book' -match 'oo'`
  - `'book' -match 'ook'`
  - `'BOOK' -match 'bo'`

Character groups

- `[character group]` allows you to match any number of characters **one time**, while `[^character group]` only matches characters NOT in the group.
- This expression returns **true** if the pattern matches `big`, `bog`, or `bug`.
  - `'big' -match 'b[iou]g'`
  - `'biggest' -match 'b[iou]g'`
  - `'bog' -match 'b[iou]g'`
  - `'bug' -match 'b[iou]g'`
  - `'bag' -match 'b[^iou]g'`
  - `'beg' -match 'b[^iou]g'`

Character ranges

- A pattern can also be a range of characters. The characters can be alphabetic `[A-Z]`, numeric `[0-9]`, or even ASCII-based `[ -~]` (all printable characters).

  this is a space!

- This expression returns true if the pattern matches any 2-digit number.
  - `42 -match '[0-9][0-9]'`
- This expression returns true if the pattern matches any 3-digit number.
  - `361 -match '[0-9][0-9][0-9]'`
- This expression returns true if the pattern matches any 3-digit number that did not have 0 as the first character
  - `444 -match '[1-9][0-9][0-9]'`

Numbers

- The `\d` character class will match any decimal digit. Conversely, `\D` will match any non-decimal digit.
- This expression returns true if it matches a server name.
  - `'Server-01' -match 'Server-\d\d'`
  - `'Server-66' -match 'Server-\d\d’`
  - `'Server-AX' -match 'Server-\D\D'`
  - `'Server-A6' -match 'Server-\D\d'`

Word characters

- The `\w` character class will match any word character `[a-zA-Z_0-9]`. To match any non-word character, use `\w`.
- This expression returns true. The pattern matches the first word character 'B'.
  - `'Book' -match '\w'` ← Lowercase w
- This expression returns true. The pattern matches non-word characters.
  - `' ' -match '\w'` ← Uppercase W
  - `'(8^(|)M' -match '\w'`
  - `'-----' -match '\w'`
  - `'Gy>mnasium' -match '\w'`

Wildcards

- The period (`.`) is a wildcard character in regular expressions. It will match any character except a newline (`\n`).
- This expression returns true. The pattern matches any 4 characters except the newline.
  - `'a1\ ' -match '....'`
  - `'454A' -match '....'`
  - `'Bill' -match '....'`

Whitespace

- Whitespace is matched using the `\s` character class. Any non-whitespace character is matched using `\S`.
  Literal space characters `' '` can also be used.
- These expression return true. The pattern uses both methods to match the space (an actual space and `\s`).
  - `' - ' -match '\s- '`
  - `' - ' -match '\s-\s'`

Quantifiers

- Quantifiers control how many instances of each element should be present in the input string.
- The following are a few of the quantifiers available in PowerShell:

```
Quantifier          Description
------------        ----------------------------------------
    *               Zero or more times.
    +               One or more times.
    ?               Zero or one time.
  {n,m}             At least n, but no more than m times.
```

- These quantifiers must come ***after*** something else.

Quantifiers

- The asterisk (*) matches **the previous element zero or more times**. The result is that even an input string without the element would be a match.
- This returns true for all account name strings even if the name is absent.
  - `'ACCOUNT NAME:    Administrator' -match 'ACCOUNT NAME:\s*\w*'`
- Breakdown of `\s*\w*`
  - `\s*` = any number of spaces
  - `\w*` = any number of word characters

Any number of spaces or zero spaces.

Any number of word characters, or zero word characters.

Quantifiers

- The plus sign (+) matches the **previous element one or more times**.
- This returns true if it matches any server name.
  - `'DC-01' -match '[A-Z]+-\d\d'`

    `\d\d` Any two numbers

    `[A-Z]+` Match one or more characters A-Z
- This returns **false** if it matches any server name.
  - `'ACCOUNT NAME:Administrator' -match 'ACCOUNT NAME:\s+\w*'`

    No space

    Must have 1 or more spaces

Quantifiers

- The question mark ? matches the **previous element zero or one time**. Like asterisk *, it will even match strings where the element is absent.
- This returns true for any server name, even server names without dashes.
  - `'SERVER01' -match '[A-Z]+-?\d\d'`

If there is one hyphen or zero hyphens

Quantifiers

- The `{n,m}` quantifier can be used several different ways to allow granular control over the quantifier. The second element `m` and the comma `,` are optional.

```
Quantifier   Description
------------ ----------------------------------------
{n}          Match EXACTLY n number of times.
{n,}         Match at LEAST n number of times.
{n,m}        Match between n and m number of times.
```

- This returns true if it matches any phone number.
  - '111-222-3333' -match '\d{3}-\d{3}-\d{4}'
- This returns false.
  - '111-222-3333' -match '\d{3}-\d{3}-\d{3}'

Anchors

- Anchors allow you to cause a match to succeed or fail based on the matches position within the input string. The caret ^ matches the start of a string, and $, which matches the end of a string.
  - The anchor matches your text at a specific position and discards unwanted characters (so it works like a `.Trim()`).
- The pattern expects the string `'fish'` to be the only thing on the line. This returns FALSE.
  - `'fishing' -match '^fish$'`
- This also returns FALSE
  - `'Geoffrey Beers' -match '^Beers'`
- These returns True
  - `'Geoffrey' -match 'Geoffrey$'`
  - `'Beers' -match '^Beers'`
- Tip: make sure you're using ' ' instead of “ ” when using anchors. If not $ will be interpreted as a variable.

Escaping characters

- The backslash (\) is used to escape characters so they aren't parsed by the regular expression engine.
  - The following characters are reserved: `[]().\^$|?*+{}`
- You'll need to escape these characters in your patterns to match them in your input strings.
- This returns true and matches numbers with at least 2 digits of precision. The decimal point is escaped using the backslash.
  - `'3.141' -match '3\.\d{2,}'`

Illegal character being escaped

`\.` Escapes `.`

`\d{2,}` At least 2 numbers

Escaping characters

- The following are a few commonly used character escapes:

```
Character Escape   Description
------------------ ----------------------------
\t                 Matches a tab
\n                 Matches a newline
\r                 Matches a carriage return
```

Where do I use these?

Select-String

- `Select-String` finds text in strings of text and files.
  - `-pattern` – The RegEx patter to match
  - `-caseSenstive` – Turn Case sensitivity on or off
-
```
gci -Filter *.log -Recurse |
    Select-String -Pattern "\s40[0-9]\s" |
        FT Filename,LineNumber,Line –Wrap
```
-
```
gci -Filter *.log -Recurse |
    Select-String -Pattern "7\.0;[\w\W]+\+Gecko" |
        FT Filename,LineNumber,Line –Wrap
```

Using -match

- We can use -match to filter text for specific string patters or number patters.
-
```
Get-EventLog Security |
     where {$_.eventID -eq 4624} |
           Select-Object -ExpandProperty Message |
                 Select-String -Pattern "\w+-GGB\$" |
                       more
```

```
Get-EventLog Security |
     where {
           $_.eventID -eq 4776 -and $_.message -match "\w+-GGB\$“
     } | more
```

More `-match` examples

- `gci | where {$_.name -match 'File\d'}`
  - Matches files with and word characters followed by a number (but no spaces)
- `gci | where {$_.name -match ‘File\s\d'}`
  - Matches files with `File`, followed by a space, followed by any digit 0 through 9
- `gci | where {$_.name -match 'File\s?\d{1,3}'}`
  - Matches a `File`, followed by zero or more spaces, followed by 1 to 3 digits
- `gci | where {$_.name -match '-'}`
  - Matches a file name that contains a hyphen
- `gci | where {$_.name -match 'File\d{1,2}\.txt'}`
  - Matches a file name that contains `File`, followed by 1-2 numbers, followed by '.txt'
- `gci | where {$_.name -match "\wFile"}`
  - Match any file with `File`, in the name and is preceded by a non-character word.